

Logs



Auditoria — Coleta Premiada

O Coleta Premiada tem efeito fiscal direto: cada registro de coleta altera a pontuação de um imóvel e, ao final do ciclo, determina um desconto real no IPTU do morador. A rastreabilidade é obrigatória por lei e por integridade do programa público.

Por que auditar?

Cenário	Situação
 Sem auditoria	Um agente corrigiu o peso de uma coleta de 2,0 kg para 12,0 kg. A pontuação do imóvel saltou para a faixa de 10% de desconto. Impossível rastrear quem fez isso, quando e com qual justificativa.
 Com auditoria	O log mostra: usuário <code>supervisor.carlos</code> , em 14/04/2026 às 10:47, alterou <code>peso_kg</code> de <code>2.0</code> para <code>12.0</code> na coleta #4821, justificativa: "erro de digitação".

Script de auditoria (PostgreSQL)

1 — Criar o schema de auditoria

```
CREATE SCHEMA IF NOT EXISTS audit;
```

2 — Criar a tabela de log

```
CREATE TABLE IF NOT EXISTS audit.sistema_audit_log (  
    id SERIAL PRIMARY KEY,  
    table_name VARCHAR(255) NOT NULL,  
    action_type VARCHAR(10) NOT NULL  
        CHECK (action_type IN ('INSERT', 'UPDATE', 'DELETE')),  
    record_id INTEGER,  
    old_data JSONB,
```

```

new_data      JSONB,
changed_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
db_user       VARCHAR(255) DEFAULT current_user
);

```

Por que JSONB?

- Uma única tabela audita todas as entidades (sem uma tabela de log por model)
- Armazena o snapshot completo antes/depois, permitindo reconstrução histórica
- Permite consultas flexíveis por campos usando operadores do PostgreSQL (`>>`, `@>`, etc.)

Exemplo de dado gravado:

```

old_data: {"peso_kg": 2.0, "material": "plastico", "status": "pendente"}
new_data: {"peso_kg": 12.0, "material": "plastico", "status": "confirmado"}

```

3 — Criar a função de auditoria

```

CREATE OR REPLACE FUNCTION audit.audit_trigger_func()
RETURNS TRIGGER AS $$
BEGIN
    -- DELETE
    IF (TG_OP = 'DELETE') THEN
        INSERT INTO audit.sistema_audit_log (
            table_name, action_type, record_id, old_data
        ) VALUES (
            TG_TABLE_NAME, TG_OP, OLD.id, row_to_json(OLD)::jsonb
        );
        RETURN OLD;

    -- UPDATE (só registra se realmente houve alteração)

```

```

    ELSIF (TG_OP = 'UPDATE') THEN
        IF ROW(OLD.*) IS DISTINCT FROM ROW(NEW.*) THEN
            INSERT INTO audit.sistema_audit_log (
                table_name, action_type, record_id, old_data,
                new_data
            ) VALUES (
                TG_TABLE_NAME, TG_OP, NEW.id,
                row_to_json(OLD)::jsonb,
                row_to_json(NEW)::jsonb
            );
        END IF;
        RETURN NEW;

    -- INSERT
    ELSIF (TG_OP = 'INSERT') THEN
        INSERT INTO audit.sistema_audit_log (
            table_name, action_type, record_id, new_data
        ) VALUES (
            TG_TABLE_NAME, TG_OP, NEW.id, row_to_json(NEW)::jsonb
        );
        RETURN NEW;
    END IF;

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

4 — Triggers por tabela

`program_imovel`

```

DROP TRIGGER IF EXISTS audit_imovel_trigger ON program_imovel;

CREATE TRIGGER audit_imovel_trigger
AFTER INSERT OR UPDATE OR DELETE ON program_imovel
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();

```

program_saldopontos

```
DROP TRIGGER IF EXISTS audit_saldopontos_trigger ON program_saldopontos;
```

```
CREATE TRIGGER audit_saldopontos_trigger  
AFTER INSERT OR UPDATE OR DELETE ON program_saldopontos  
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();
```

collection_registrocoleta

```
DROP TRIGGER IF EXISTS audit_registrocoleta_trigger ON collection_registrocoleta;
```

```
CREATE TRIGGER audit_registrocoleta_trigger  
AFTER INSERT OR UPDATE OR DELETE ON collection_registrocoleta  
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();
```

accounts_usuario

```
DROP TRIGGER IF EXISTS audit_usuario_trigger ON accounts_usuario;
```

```
CREATE TRIGGER audit_usuario_trigger  
AFTER INSERT OR UPDATE OR DELETE ON accounts_usuario  
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();
```

5 — Teste (INSERT / UPDATE / DELETE)

```
-- INSERT de teste  
INSERT INTO accounts_usuario (  
    password, is_superuser, email, cpf, nome, perfil, ativo, is_staff  
) VALUES (  
    'senha_falsa_123', FALSE, 'morador.teste.auditoria@email.com',  
    '999999999999', 'Morador de Teste Auditoria', 'morador',
```

```
TRUE, FALSE
);

-- UPDATE de teste
UPDATE accounts_usuario
SET nome = 'Morador Editado (Auditoria)', ativo = FALSE
WHERE email = 'morador.teste.auditoria@email.com';

-- DELETE de teste
DELETE FROM accounts_usuario
WHERE email = 'morador.teste.auditoria@email.com';
```

Verificar o log:

```
SELECT *
FROM audit.sistema_audit_log
ORDER BY id DESC;
```